

CSC3060 Project 4 Report

Student Name Chaoyi, Sun

Student ID 124090550

1 Implementation Summary

1.1 Task1

Task 1 requires the implementation of core access logic and fundamental mechanisms for a single-level L1 cache simulator. The completed implementation consists of three core components: address decomposition, cache access logic, and the baseline replacement policy.

- **Address Decomposition and Reconstruction**

Based on the provided cache geometry, including cache size, associativity, and block size. Bitwise shift and masking operations are employed to accurately extract the set index and tag from a 64-bit memory address.

- **Cache Access Logic**

For a cache access request, if a matching tag is found in the target set, the metadata of the hit line is updated; otherwise, the requested block is installed into an empty invalid line if one is available, or a victim block is selected by the replacement policy for eviction when the set is full.

- **LRU Replacement Policy**

Uses timestamps to track access order. On replacement, selects the cache line with the earliest timestamp as the victim.

1.2 Task2

Task 2 involves the integration of an L2 cache and the construction of a multi level memory hierarchy where L1, L2, and Main Memory are connected in sequence.

- **Hierarchy Hookup**

When the `--enable-l2` flag is active, an L2 cache is instantiated and positioned between L1 and Main Memory. The `next_level` pointer of L1 is updated to point to L2, creating a chain where L1 misses are forwarded to the secondary cache rather than directly to Main Memory.

- **Recursive Miss Path**

L1 misses trigger a search in L2, and Main Memory is accessed only after an L2 miss. This hierarchy reduces total cycles by serving a significant portion of L1 misses at a lower L2 latency.

- **Write Back Propagation**

Dirty blocks evicted from L1 are written into L2. If those blocks are subsequently evicted from L2, the modified data is propagated to Main Memory to maintain consistency across the entire hierarchy.

1.3 Task3

Task 3 optimizes the cache simulator with advanced replacement policies and prefetching mechanisms to minimize AMAT on a personalized trace.

- **Advanced Replacement Policies**

SRRIP assigns a re-reference priority value to each cache line, while BIP inserts most new lines with low priority. Both policies effectively resist scan-induced cache pollution.

- **Prefetching Strategies**

NextLine prefetches the subsequent cache block on each access. Stride detects repeating access patterns and prefetches future blocks at the computed stride once the pattern is confirmed.

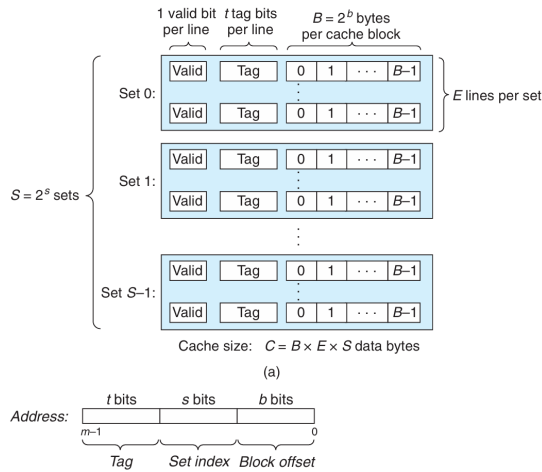
- **Prefetch Integration**

Prefetching is triggered by demand requests to load predicted blocks into the cache as clean entries. Successful prefetches are promoted to standard lines upon a subsequent hit, allowing the replacement policy to recognize and prioritize useful data.

2 Address Mapping Explanation

2.1 Calculation Methods

Given a cache configuration with total size C (in KB), associativity E , and block size B (in bytes). Let $S = \frac{1024C}{BE}$ be the number of sets in the cache.



Parameter	Formula
Block offset bits	$\log_2 B$
Set index bits	$\log_2 S$
Tag bits	$64 - \log_2 B - \log_2 S$

2.2 Impact of Cache Geometry Changes

- Increasing block size increases offset bits but decreases index bits while leaving tag bits unchanged.
- Increasing cache size increases index bits and decreases tag bits.
- Increasing associativity decreases index bits and increases tag bits.

Change	Block Offset Bits	Set Index Bits	Tag Bits
Increase block size B	↑	↓	—
Increase cache size S	—	↑	↓
Increase associativity E	—	↓	↑

3 Task 1 Testing

3.1 Testing Methodology

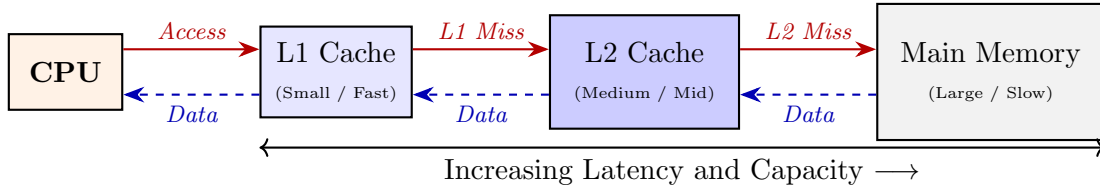
The simulator was tested on `trace_sanity.txt` using diverse cache geometries including various sizes, associativity levels, and block sizes. These tests confirmed that the dynamic address decomposition and LRU policy correctly handle frequent misses in small configurations as well as corner cases like direct-mapped caches. All output metrics such as hit rate, write-back counts, and AMAT were cross-checked against reference results to ensure the absolute accuracy of the core access logic.

3.2 Test Configurations

Test Category	L1 Size	Assoc	Block	Hit Rate	Misses	WB	Mem Acc	AMAT
Basic Configs	16 KB	4	64 B	21.43%	44	6	50	79.57
	32 KB	4	32 B	21.43%	44	2	46	79.57
	64 KB	8	128 B	60.71%	22	0	22	40.29
Direct Mapped	32 KB	1	64 B	42.86%	32	2	34	58.14
High Associativity	64 KB	32	64 B	58.93%	23	0	23	42.07
Large Capacity	128 KB	4	64 B	62.50%	21	0	21	38.50
Minimum Capacity	1 KB	2	64 B	21.43%	44	8	52	79.57

4 Task 2 Hierarchy Explanation

4.1 L1, L2, and Memory Interaction



As illustrated in the figure, memory requests originate at the CPU and traverse the hierarchy toward Main Memory. A hit at any cache level triggers an immediate data return along the response path. In the case of an L1 miss, the request is forwarded to L2, and a subsequent L2 miss results in a Main Memory access. Once the data is retrieved, it flows back through each level to satisfy the CPU request. Simultaneously, dirty victims follow an eviction path where L1 writes back modified data to L2, and L2 propagates its own dirty blocks to Main Memory to maintain data consistency.

4.2 Impact of L2 on System Performance

- **Filtering of Main Memory Traffic**

Main memory accesses dropped from 46 to 23 in the reference trace. This reduction occurs because the L2 cache acts as a secondary filtering layer. Every L1 miss that results in an L2 hit is successfully served within the cache hierarchy, preventing a high latency request from reaching Main Memory.

Average Memory Access Time Optimization AMAT decreased from 79.57 to 45.21 cycles. Although L1 misses now incur additional L2 access latency, this overhead is negligible compared to the much higher memory penalty that L2 helps avoid. With L2 capturing 50% of L1 misses, the average data access time is significantly reduced.

- **Write Back Staging and Buffer Effect**

The write back process is now staged between levels. Dirty evicts from L1 are written into L2 instead of directly to Main Memory. This creates a buffer effect where modified data can stay within the faster cache hierarchy for a longer duration.

5 Task 3 Design Choices

5.1 Replacement Policies

Three replacement policies are implemented: LRU, SRRIP, and BIP. LRU serves as the baseline. SRRIP assigns a re-reference priority value to each cache line, protecting hot data from scan-induced pollution. BIP is implemented but not used in the final configuration, as SRRIP provides better protection for the trace’s mixed phase behavior.

5.2 Prefetchers

Three prefetchers are implemented: NextLine, Stride, and a custom prefetcher. NextLine prefetches the subsequent cache block, benefiting sequential patterns. Stride detects repeating strides and prefetches future blocks once the pattern is confirmed.

The custom prefetcher is designed based on the personalized trace. Analysis reveals that certain strides dominate the access pattern. Therefore, when such strides are detected, the custom prefetcher issues an additional prefetch to further reduce miss penalty.

6 Trace Analysis

6.1 Trace Generation

The personalized trace was generated using student ID **124090550** as the seed:

```
./trace_generator/workload_gen student 124090550 > my_trace.txt
```

6.2 Access Patterns

The trace was analyzed using the provided `trace_analyzer.py` script with a block size of 64 bytes.

Feature	Value	Implication
Dominant Strides	7 (45%), 1 (25%), 64 (17%)	Use Stride prefetcher for L1
Hotspot Set	Set 0 (22.65% of accesses)	Use high associativity (32-way)
Phase Behavior	Stable loop [2560,4352): 14 blocks, 1 set	Use SRRIP to protect hot data
L1 Miss Pattern	Scan phases: sequential new blocks	Use NextLine prefetcher for L2

Based on these observations, stride 7 (45%), stride 1 (25%), and stride 64 (17%) together guided the use of Stride prefetcher in L1. The hotspot on set 0 (22.65%) demanded 32-way associativity. The stable loop phase (14 blocks, 1 set) required SRRIP in L2 to protect hot data from scan pollution. Sequential L1 misses during scan phases made NextLine prefetcher effective for L2.

7 Experimental Results

Table 1: Single-Level L1 Performance Comparison (32KB, 8-way)

Policy	Prefetcher	L1 Hit Rate	AMAT (cycles)
LRU	None	16.17%	84.83
	NextLine	74.98%	26.02
	Stride	86.66%	14.34
SRRIP	None	20.83%	80.17
	NextLine	69.32%	31.68
	Stride	88.20%	12.80
BIP	None	28.20%	72.80
	NextLine	43.42%	57.58
	Stride	88.05%	12.95

Table 2: Full Comparison of Cache Configurations (32KB, 8-way)

L1 Policy	L1 Pref.	L2 Policy	L2 Pref.	L1 Hit%	L2 Hit%	AMAT
LRU	NextLine	LRU	NextLine	74.98%	98.07%	4.18
			Stride	74.98%	94.96%	4.18
			NextLine	74.98%	98.06%	4.18
		SRRIP	Stride	74.98%	94.78%	4.33
			NextLine	74.98%	98.03%	4.18
			Stride	74.98%	94.80%	4.34
LRU	Stride	LRU	NextLine	86.66%	97.54%	1.85
			Stride	86.66%	98.07%	2.14
			NextLine	86.66%	97.53%	1.86
		SRRIP	Stride	86.66%	97.97%	2.20
			NextLine	86.66%	97.53%	1.86
			Stride	86.66%	97.96%	2.21
SRRIP	NextLine	LRU	NextLine	69.32%	98.11%	4.40
			Stride	69.32%	92.56%	4.40
			NextLine	69.32%	98.11%	4.40
		SRRIP	Stride	69.32%	92.37%	4.55
			NextLine	69.32%	98.11%	4.40
			Stride	69.32%	92.22%	4.55
SRRIP	Stride	LRU	NextLine	88.20%	97.41%	1.78
			Stride	88.20%	98.00%	2.06
			NextLine	88.20%	97.39%	1.80
		SRRIP	Stride	88.20%	97.88%	2.14
			NextLine	88.20%	97.39%	1.80
			Stride	88.20%	97.85%	2.16
BIP	NextLine	LRU	NextLine	43.42%	98.38%	5.44
			Stride	43.42%	90.26%	5.44
			NextLine	43.42%	98.38%	5.44
		SRRIP	Stride	43.42%	90.17%	5.44
			NextLine	43.42%	98.38%	5.44
			Stride	43.42%	90.15%	5.44
BIP	Stride	LRU	NextLine	88.05%	97.31%	1.79
			Stride	88.05%	97.92%	2.07
			NextLine	88.05%	97.30%	1.80
		SRRIP	Stride	88.05%	97.83%	2.14
			NextLine	88.05%	97.30%	1.80
			Stride	88.05%	97.81%	2.15

8 Best Configuration and Discussion

Table 3: Progressive Optimization Results

Configuration	Baseline	Best 8-way	Tunable Params	Optimized
L1 Associativity	8-way	8-way	32-way	32-way
L1 Policy	LRU	SRRIP	LRU	LRU
L1 Prefetcher	None	Stride	Stride	Custom
L2 Associativity	8-way	8-way	32-way	32-way
L2 Policy	LRU	LRU	SRRIP	SRRIP
L2 Prefetcher	None	NextLine	NextLine	NextLine
L1 Hit Rate	16.17%	88.20%	97.18%	99.04%
L2 Hit Rate	77.41%	97.41%	96.84%	96.81%
AMAT (cycles)	23.81	1.78	1.43	1.23
Final Improvement: 94.8%				

Based on the analysis of the personalized trace generated with my student seed, the design is specifically tuned to its observed characteristics, particularly the dominant strides 7 (45%), 1 (25%), and 64 (17%). The Custom prefetcher aggressively exploits these patterns to achieve high performance. Consequently, the robustness of this configuration is limited, and it may fail on workloads with different access patterns.

9 Appendix

It is hereby declared that Generative AI tools were utilized in this project to assist with code optimization, report polishing, and test case generation. The core logic and implementation were verified by the author.

Link